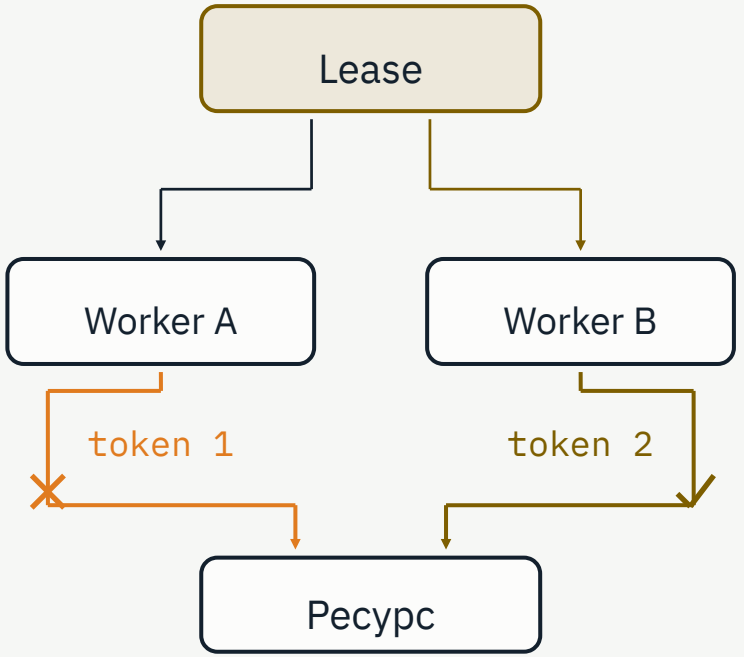
ПРИМЕЊЕНО СОФТВЕРСКО ИНЖЕЊЕРСТВО

ОСНОВЕ ДИСТРИБУИРАНОГ ПРОГРАМИРАЊА

Координација, проток и евентуална конзистентност

Јасни компромиси при расту система



Факултет техничких наука · Универзитет у Новом Саду

Нови Сад

ПРОВЕРА

Садржај презентације

01

Појмови и границе

Кључни појмови, одговорности компоненти и конкретан failure сценарио.

03–10

02

Дистрибуирани ток

Шема тока и гаранције које повезују компоненте система.

11–13

03

Код и failure тестови

Покретање примера, доменско правило, тест и видљиви исходи.

14–18

04

Примена и провера

Принципи, замке, задатак уз пример и примена на пројектну целину.

19–26

ПРВИ ДЕО

01

Појмови и границе

Кључни појмови, одговорности компоненти и конкретан failure сценарио.

01 · ПОЈМОВИ

Тема и проблем

Објаснити како се бирају гаранције власништва, оптерећења и свежине података.

Више инстанци, већи улазни ритам и реплике читања уводе сукоб, заостајање и привремено различите приказе стања.

НАПОМЕНА

Координација штити ексклузиван рад, backpressure спречава колапс, а реплике убрзавају читање уз цену застарелости.

01 · ПОЈМОВИ

Четири појма теме

Напредни дистрибуирани дизајн прихвата да различите компоненте могу привремено имати различит поглед на стање.

01 **Власништво**

један важећи извршилац

02 **Оптерећење**

ритам производње и обраде

03 **Реплика**

брже читање уз кашњење

04 **Јасан уговор**

видљива ограничења

01 · ПОЈМОВИ

Важећи извршилац

Власништво бира инстанцу чија одлука тренутно важи. Lease, epoch или fencing токен спречавају закашњелог власника да пише након промене режима.

01 Само једна инстанца има важећу одлуку у датом тренутку.

02 Fencing токен спречава закашњелог власника да пише.

03 Кратак прекид не сме сам по себи покренути failover.

01 · ПОЈМОВИ

Оптерећење и проток

Проток пореди брзину пријема и обраде. Backpressure уводи границу пре него што ред и меморија прерасту капацитет система.

01 Пореди брзину пријема и обраде.

02 Backpressure поставља границу пре колапса.

03 Неограничен ред само одлаже и скрива проблем.

01 · ПОЈМОВИ

Реплика за читање

Реплика убрзава читање, али може приказати старије стање од ауторитативног извора. То је прихватљиво само када је кориснику јасно шта гаранција значи.

01 Убрзава читање уз цену могућег заостајања.

02 Корисник мора знати да приказ може бити застарео.

03 Не сме се тумачити као ауторитативан извор.

01 · ПОЈМОВИ

Уговор о ограничењу

Уговор објашњава када резултат може бити застарео и када систем мора чекати ауторитативно стање. Тиме компромис постаје свестан дизајнерски избор.

01 Јасно каже када је резултат дозвољено да буде застарео.

02 Каже и када се мора чекати ауторитативно стање.

03 Компромис је свестан избор, не случајна последица.

Пример: два чвора и један ексклузиван посао

Две инстанце накратко изгубе везу једна са другом.

01 Координација бира једног важећег извршиоца за рад који не сме бити паралелан.

02 Backpressure одлучује како систем реагује када улазни ритам надмаши обраду.

03 Реплика убрзава читање, али њен приказ мора моћи да покаже застарелост.

ДРУГИ ДЕО

02

Дистрибуирани ток

Шема тока и гаранције које повезују компоненте система.

02 · ТОК

Шема рада под притиском

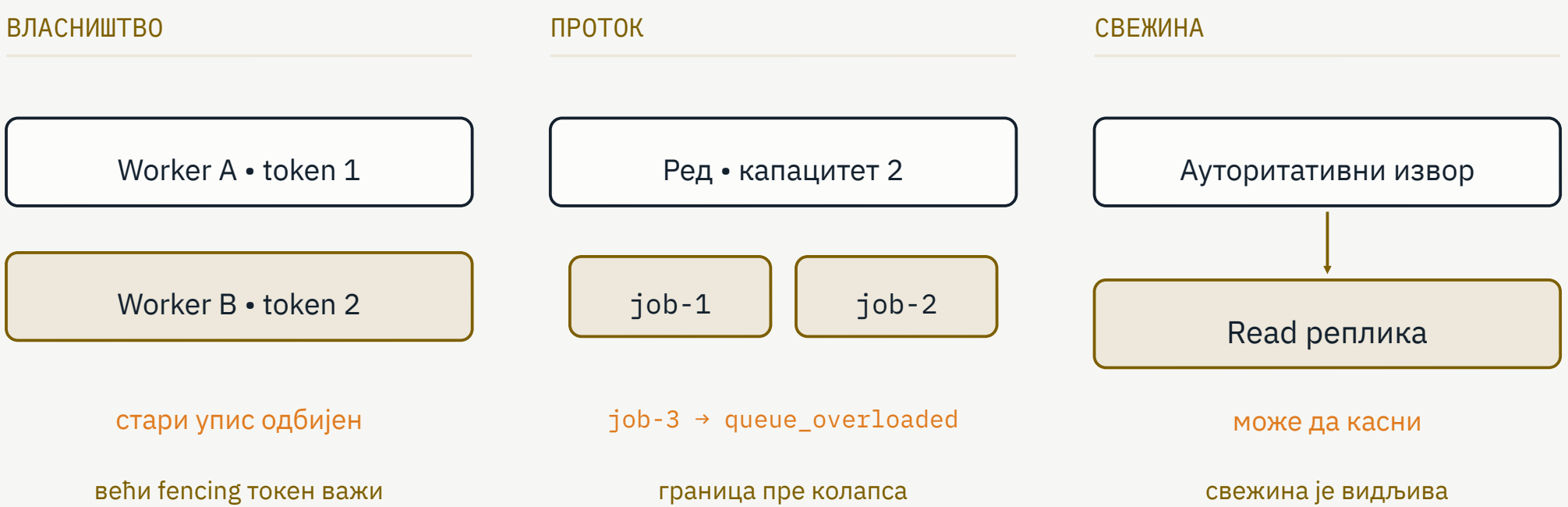
Напредни дистрибуирани дизајн не скрива компромис, већ кориснику и оператеру јасно показује гаранцију.

Елемент	Улога у току
Власништво	један важећи извршилац
Оптерећење	контролисан ритам обраде
Реплика	брже читање уз кашњење
Јасан уговор	видљиве границе гаранције

02 · ДИЈАГРАМ

Три гаранције које систем мора да изабере

Fencing штити важећег власника, ограничен ред чува капацитет, а реплика показује свежину.



ТРЕЋИ ДЕО

03

Код и failure тестови

Покретање примера, доменско правило, тест и видљиви исходи.

Покретање примера

Пример показује две гаранције које систем мора да изабере када расте.

terminal · Корен Git репозиторијума nastava.

```
cd "03 Osnove Distribuiranog Programiranja/04 Primeri/08 Koordinacija protok"
dotnet run --project src/Odp.Vezba08.ConsoleUi
dotnet test Odp.Vezba08.sln
```

03 · КОД И ТЕСТ

Правило у коду

У једном тренутку само један worker је власник ресурса.

C# · доменско правило

```
1   public LeaseDecision Acquire(Lease? current, string resource, string candidateId, DateTimeOffset now)
2   {
3       ArgumentException.ThrowIfNullOrWhiteSpace(resource);
4       ArgumentException.ThrowIfNullOrWhiteSpace(candidateId);
5       if (current is null || now >= current.ExpiresAt)
6       {
7           var token = (current?.FencingToken ?? 0) + 1;
8           return new(true, LeaseCodes.Granted, new Lease(resource, candidateId, token, now +
9               _duration));
10      }
11      return current.OwnerId == candidateId
12          ? new(true, LeaseCodes.Renewed, current with { ExpiresAt = now + _duration })
13          : new(false, LeaseCodes.HeldByAnother, current);
14  }
```

03 · КОД И ТЕСТ

Тест који задаје квар

После истека lease-а нови власник добија већи токен, а упис бившег власника је одбијен.

C# · NUnit

```
1   [Test]
2   public void Write_WhenFormerOwnerWakesUpAfterTakeover_IsRejectedByItsStaleToken()
3   {
4       var former = _acquire.Acquire(Resource, "worker-a").Lease;
5       _clock.Advance(TimeSpan.FromSeconds(45));
6       var current = _acquire.Acquire(Resource, "worker-b").Lease;
7       _write.Write(Resource, current.FencingToken, "plan B");
8       var code = _write.Write(Resource, former.FencingToken, "zakasneli plan A");
9       using (Assert.EnterMultipleScope())
10      {
11          Assert.That(code, Is.EqualTo(FencingCodes.StaleToken));
12          Assert.That(_schedules.ValueOf(Resource), Is.EqualTo("plan B"));
13      }
14  }
```

Испис демонстрације

Када је ред пун, пошиљалац добија queue_overloaded уместо тихог чекања.

console · исходи демонстрације

```
Koordinacija:
  worker-a traži lease -> lease_granted (vlasnik worker-a, token 1)
  worker-b traži lease -> lease_held_by_another (vlasnik worker-a, token 1)
  worker-a upisuje "plan A" sa tokenom 1 -> write_accepted
  worker-a je zastao 45 s; lease je istekao
  worker-b traži lease -> lease_granted (vlasnik worker-b, token 2)
  worker-b upisuje "plan B" sa tokenom 2 -> write_accepted
  worker-a upisuje "zakasneli plan A" sa tokenom 1 -> stale_fencing_token
  važeći raspored: plan B
Protok (kapacitet reda 2):
  job-1 -> job_accepted
  job-2 -> job_accepted
  job-3 -> queue_overloaded
```

ЧЕТВРТИ ДЕО

04

Примена и провера

Принципи, замке, задатак уз пример и примена на пројектну целину.

04 · ПРИМЕНА

Принципи и замке

Принцип важи тек када је препозната и замка која га тихо поништава.

ПРИНЦИПИ

- Fencing токен спречава да стари власник настави рад након преузимања.
- Backpressure бира контролисано успоравање или ограничен губитак пре потпуног колапса.
- Read модел може бити застарео, али систем мора показати колико и када то утиче на одлуку.

ЗАМКЕ

- Failover покренут на основу једног кратког прекида.
- Неограничен ред порука који само одлаже квар.
- Коришћење застарелог read модела као да је ауторитативно стање.

04 · ПРИМЕНА

Типичне грешке

Ове грешке се најчешће понављају када се принцип површно примени.

01 Failover због кратког прекида.

02 Неограничен ред који само скрива проблем.

03 Застарео приказ се тумачи као ауторитет.

04 · ПРИМЕНА

Како се принцип примењује

Кораци показују како се принцип препознаје и доследно спроводи у развоју.

01 Одредити шта захтева најновије стање.

02 Увести прагове за промену режима рада.

03 Спречити закашњелог власника јасним токеном или епохом.

04 · ПРИМЕНА

Питање за дискусију

Одговор се образлаже failure сценаријем и тестом, а не описом срећног пута.

За коју одлуку ваш систем може користити застарео приказ, а за коју мора чекати ауторитативно стање?

> ЗА ВЕЖБУ
> За једну одлуку наведите да ли захтева најновије стање или може да користи застарео приказ, па објасните последицу избора.

04 · ПРИМЕНА

Задатак уз пример

Проширење примера уз вежбу, правило иде у домен, а доказ у тест.

01 Додати у приказ распореда податак о свежини: време последњег уписа и ознаку IsStale када је приказ старији од задатог прага.

02 Тестом показати да читалац види застарео, али јасно означен податак док власник не упише нови.

04 · ПРИМЕНА

Контролна тачка P4 (26.05.)

Последња контролна тачка (Вежба 7–8) проверава оперативни и напредни ниво: телеметрију, команде, disconnected токове и координацију.

01 Telemetry и command/job токови имају документовану delivery семантику.

02 Disconnected station ток, dead-letter/replay и transactional outbox/inbox су имплементирани.

03 Најмање три евалуациона/дистрибуирана сценарија, укључујући најмање један failure/recovery случај.

04 Сваки члан тима објашњава свој део без ослањања на аутоматски генерисан одговор.

04 · ПРИМЕНА

Резиме

Завршна одбрана не тражи савршену архитектуру.

Напредни дистрибуирани дизајн не скрива компромис, већ кориснику и оператеру јасно показује гаранцију.

НАПОМЕНА

Завршна одбрана обухвата све теме и додељену пројектну целину.